Chapter 7

Exploitation of problem-specific features

Chapter 7. Exploitation of problem-specific features

7.1 Introduction

7.2 Problem Decomposition

7.3 Recognition and Searching in k-trees

7.4 Problem Reduction by Removing Redundant Constraints

7.5 Cycle-cutsets, Stable Sets and Pseudo_Tree_Search

7.6 The Tree-clustering Method

7.7 j-width and Backtrack-bounded Search

7.8 CSPs with Binary Numerical Constraints

7.1 Introduction

If there are n variables in a CSP, and the maximum size of the domains of the variables is a, one would have to deal with a search tree with $O(a^n)$ leaves. Therefore, worst case time complexity of CSP solvers is $O(a^n)$ in general. In this chapter, we shall look at techniques which exploit the specific features of the individual problems and hopefully reduce the time complexity to below $O(a^n)$.

1. The possibility of decomposing problems into **independent sub-problems** (which allows one to apply the divide and conquer strategy).

2. A set of "easy problems", namely those in which constraint graphs form **trees and *k*-trees**, for which efficient algorithms exist.

3. Techniques to **remove redundant constraints** to transform CSPs to equivalent but "easy" problems.

4. **The cycle-cutset method**, a dynamic search method that switches to a backtrack-free search when the remaining problem is easy.

5. **The tree-clustering method**, which groups the variables into clusters to form sub-problems and solves the problem by solving these smaller and easier sub-problems separately.

6. Extend the **relationship between** the width of a constraint graph and *k*-consistency.

7. Introduce specialized algorithms for handling CSPs with **numerical variables and conjunctive binary constraints**.

7.2 Problem Decomposition

If the variables in a CSP can be separated into independent groups (i.e. no variable in one group constrains any variable in any other group), then these groups of variables can be labelled separately. When this is the case, a smaller space needs to be searched.

A problem can be decomposed if its primal graph is not connected. A graph with n nodes can be partitioned in O(n) time, using the Partition procedure below. Therefore, running a graph partitioning algorithm before searching for solutions does not increase the overall complexity of the search algorithm.

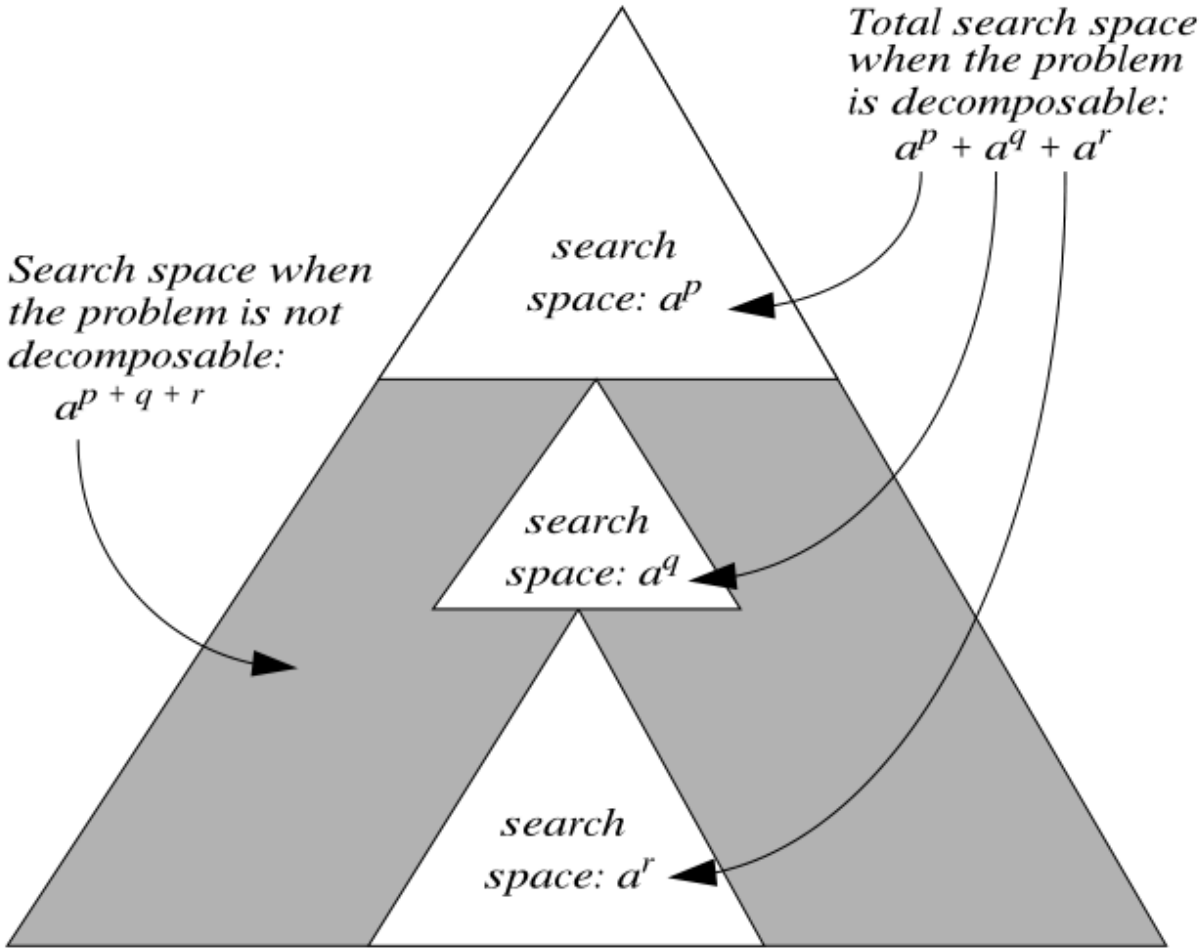


Figure 7.1 The size of the search space when a problem is decomposable. a = size of the domains of the variables and the total number of variables in the problem = $p + q + r$

```
PROCEDURE Partition(V, E)
/* given a graph, partition the nodes into unconnected clusters */
BEGIN
    SS ← { };    /* SS is the set of clusters of variables to be returned */
    WHILE (V ≠ { }) DO
        BEGIN
            z ← any node in V; S ← {z}; V ← V – S; Cluster ← { };
            /* S and Cluster are used as working storage */
            WHILE (S ≠ { }) DO
                BEGIN
                    x ← any element in S; S ← S – {x};
                    Cluster ← Cluster + {x};
                    FOR each y such that (x,y) is in E AND y is in V DO
                        BEGIN
                            V ← V – {y};
                            S ← S + {y};
                            E ← E – {(x,y)};
                        END
                END /* of inner WHILE loop */
            SS ← SS + Cluster;              /* one cluster found */
        END; /* of outer WHILE loop */
    return(SS);
END /* of Partition */
```

7.3 Recognition and Searching in *k*-trees

7.3.1 "Easy problems": CSPs which constraint graphs are trees

When the constraint graph (Definition 4-1) of a CSP is a tree, one can solve this problem in $O(na^2)$, where n is the number of variables and a is the maximum domain size in the problem. This motivates the recognition of problems which constraint graphs are trees, or reducing CSPs to problems of such class.

A tree is a graph with a width equal to 1. If the constraint graph of a CSP is a tree, then a search for solutions in this problem is backtrack-free if node- and arc-consistency (i.e. strong 2-consistency) are maintained in it. In fact, strong 2-consistency is stronger than necessary to guarantee a search to be backtrack-free in a CSP which constraint graph is a tree. All one needs to achieve is DAC in the CSP.

```
PROCEDURE Tree_search((Z, D, C))
/* The constraint graph of (Z, D, C), G((Z, D, C)), is a tree */
BEGIN
    Give the variables an ordering < such that all parents are placed
        before their children in G((Z, D, C));
    achieve NC and DAC in (Z, D, C, <);
    Labelled ← { };
    /* backtrack-free search */
    WHILE Z ≠ { } DO
        BEGIN
            x ← the frontmost variable in Z according to <;
            Z ← Z – {x};
            v ← a value in D_x such that <x,v> is compatible with all the
                labels in Labelled;
            Labelled ← Labelled + {<x,v>};
        END;
    return( Labelled );
END /* of Tree_search */
```

The procedure Acyclic recognizes acyclic undirected graphs (i.e. trees) in $O(n)$, where n is the number of variables in the problem.

```
PROCEDURE Acyclic(V, E)
/* Return True if the graph (V, E) is acyclic, return False otherwise */
BEGIN
    WHILE (V ≠ { }) DO
        BEGIN
            y ← any node in V;
            S ← {y};
            WHILE (S ≠ { }) DO
                BEGIN
                    z ← any node in S; S ← S – {z};
                    V ← V – {z};
                    FOR each x adjacent to z with regard to E DO
                                        /* (x,z) ∈ E */
```

```
                    BEGIN
                        IF (x is in S) THEN return(False);
                        E ← E – {(x, z)};
                        /* note that (x, z) is the same object as (z, x) */
                        S ← S + {x};
                    END
                END
            END
        return(True);
    END /* of Acyclic */
```

The time complexity of Acyclic is $O(n)$.

7.3.2 Searching in problems which constraint graphs are *k*-trees

Let n be number of variables in the problem and a be the maximum size of the domains. If the constraint graph of a problem can be recognized as a k-tree, then it can be solved in $O(na^{k+1})$ time. This can be achieved by first finding an ordering which induced-width is k, and then achieving adaptive-consistency in the problem.

The procedure W which, given a graph and an integer k, determines **whether the graph is a *k*-tree**, and **returns an ordering of the nodes** such that the induced-width of the graph equals k.

```
PROCEDURE W( (V, E), k );
/* Given a constraint graph G = (V, E) of a constraint satisfaction
   problem P and an integer k, return an ordering < of V such that
   induced-width(P, <) = k if G is a k-tree; NIL if it is not */
BEGIN
   /* initialization */
  K ← { }; Sum ← 0;
  FOR each node x in E DO
      BEGIN
          Count[x] ← the degree of x;
          IF (Count[x] = k) THEN K ← K + {x};
          Sum ← Sum + Count[x];
      END
  IF (Sum ≠ 2nk – k – k^2) THEN return(NIL);
```

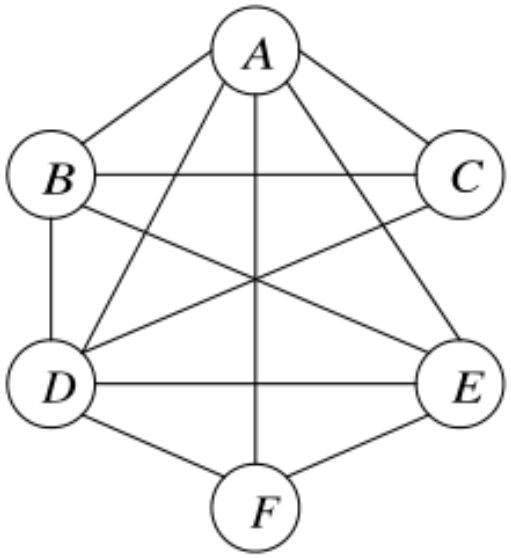
```
    /* major computation */
    FOR i = n to k + 1 by −1 DO
        IF (K = { }) THEN return(NIL);
        ELSE BEGIN
                        v ← any node in K; K ← K − {v};
                        V' ← neighbourhood(v);
                        E' ← {(a,b) | (a,b) ∈ E ∧ a, b ∈ V'};
                        IF ((V', E') is a complete graph) THEN
                            BEGIN
                                Ordering[i] ← v; V ← V − {v};
                                FOR each w such that (v, w) ∈ E DO
                                    BEGIN
                                        E = E − {(v, w)};
                                        Count[w] ← Count[w] − 1;
                                        IF (Count[w] = k) THEN K ← K + {w};
                                    END;
                            END
                        ELSE return(NIL);
                END /* of ELSE */
    /* at this point, all but k nodes have been ordered */
    IF (the remaining k nodes form a complete graph)
    THEN BEGIN
            Ordering[1] to Ordering[k] ← remaining nodes in any order;
            return(Ordering);
        END
    ELSE return(NIL);
END /* of W */
```

The procedure W is in fact providing a constructive proof to the proposition that the input graph is a k-tree.

According to the definition, a k-tree should have exactly $k(k-1)/2 + (n-k)k$ edges, where n is the number of nodes in the k-tree. This is because a k-tree must contain a complete graph of k nodes, which has $k(k-1)/2$ edges. Besides the nodes in this complete graph, there should be $(n-k)$ other nodes, each of them having exactly k edges (according to the definition), making $(n-k)k$ edges in total. So the total number or edges in a k-tree is $k(k-1)/2 + (n-k)k$.

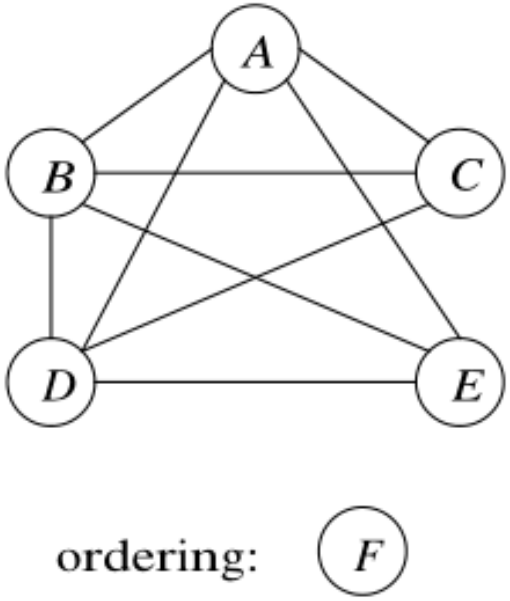
It should be noted that the ordering of the nodes in the set K is unimportant. This is because if the graph is indeed a k-tree, then no two nodes in K should be adjacent to each other (otherwise the removal of one of them would cause the degree of the other to be reduced below k).

The whole procedure W takes $O(n)$ time to compute.



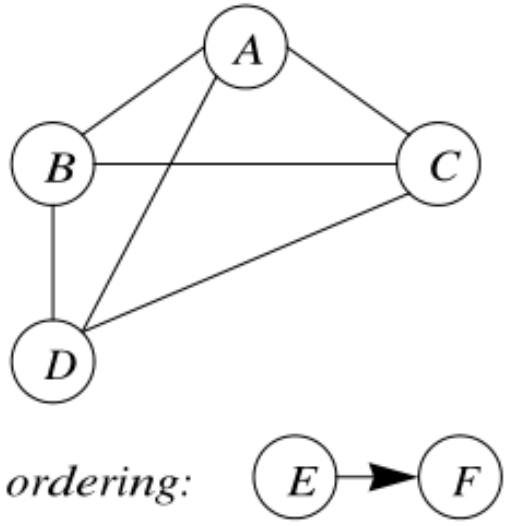
$K = \{C, F\}$

(a) An input graph to be recognized as a 3-tree (value of K after initialization of procedure W)

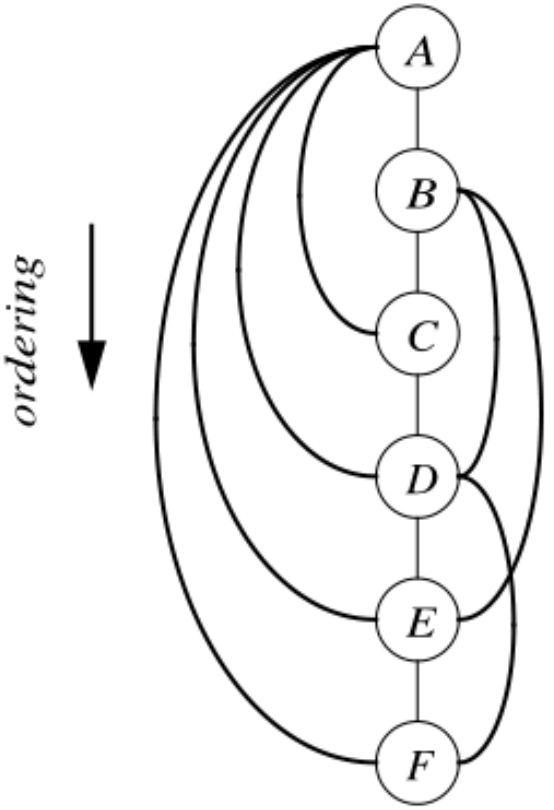


$K = \{C, E\}$

(b) The graph, ordering and K after the first step of procedure W



(c) The graph, ordering and K after the second step of procedure W (any node can be chosen in the next step, which is skipped)



(d) Ordering produced by W (achieving adaptive-consistency will not add extra edges to it)

Theorem 7.2

A k-tree constraint graph with n nodes can be recognized as a k-tree, and an ordered constraint graph with induced-width k can be found (or k-1 for trivial k-trees), in O(n) time.

Theorem 7.3

Let G be the primal graph of a CSP P*. If G is a trivial k-tree, then the induced-width of* P *is* $k - 1$*. If G is a non-trivial k-tree, then the induced-width of* P *is k. Whenever G is a k-tree, the induced-width of* P *is equal to G's width:*

$\forall$ csp(P): trivial_k-tree(G(P)) $\Rightarrow$ induced-width(P) = $k - 1$

$\forall$ csp(P): k-tree(G(P)) $\Rightarrow$ induced-width(P) = k

$\forall$ csp(P): induced-width(G(P)) = width(G(P))

The k-tree_search procedure below shows one way of exploiting the fact that the CSP's constraint graph is a k-trees.

```
PROCEDURE k-tree_search(Z, D, C)
/* given a CSP, finds a solution to it in O(n^(ak+1)) time if its constraint
    graph is a k-tree for some k; otherwise, return NIL */
BEGIN
    /* check if the constraint graph is a k-tree for any k */
    k ← 1;
    REPEAT
        Ordering ← W((Z, D, C)), k);
        k ← k + 1;
    UNTIL (Ordering ≠ NIL) OR (k > |Z|);
    /* one might want to further limit the value of k above */
    IF (Ordering = NIL)
    THEN return(NIL)      /* other methods needed to solve the CSP */
    ELSE BEGIN
                P ← Adaptive_consistency(Z, D, C, Ordering);
                Result ← perform backtrack-free search on P;
                return(Result);
         END
END /* of k-tree_search */
```

Theorem 7.4

A CSP which constraint graph is a k-tree can be solved in $O(na^{k+1})$ time and $O(na^{k})$ space, where n is the number of variables in the problem, and a is the maximum domain size in the problem.

7.4 Problem Reduction by Removing Redundant Constraints

```
PROCEDURE Path_redundant((x ,y), P)
/* P is a CSP and x, y are two variables in it */
BEGIN
    (Z, D, C) ← NC-1(P);          /* achieve node-consistency in P */
    U ← {(<x,a><y,b>) | a ∈ D_x ∧ b ∈ D_y};
    FOR each z in Z such that x ≠ z AND y ≠ z DO
        BEGIN
            DisAllowed_CLs ← U – C_x,y
            For each cl ∈ DisAllowed_CLs DO
                IF NOT Permitted(cl, z, D_z, C_x,z, C_z,y)
                THEN C_x,y ← C_x,y + {cl};
            IF C_x,y = U THEN return(True)
        END
    return(False);
END /* of Path_redundant */
```

```
PROCEDURE Permitted((<x,a><y,b>), z, D_z, C_x,z, C_z,y)
BEGIN
    FOR each c ∈ D_z DO
        IF (satisfies((<x,a><z,c>), C_x,z) AND
                satisfies((<z,c><y,b>), C_z,y))
        THEN return(True);
    return(False);
END /* of Permitted */
```

It is possible to reduce a CSP to an "easy problem" (as defined in the last section) by removing redundant constraints. A constraint in a binary CSP can be removed if it is *path-redundant*— if S is the set of all *path-induced* compound labels for x and y, then the constraint $C_{x,y}$ is redundant if S is a subset of $C_{x,y}$.

Path_redundant checks whether every compound label which is disallowed by $C_{x,y}$ is indeed disallowed by at least one path. Only if this is the case could $C_{x,y}$ be deleted without relaxing the constraints in the problem.

The time complexity of Path_redundant is $O(na^3)$.

No one suggests that all the constraints in the problem should be examined. One should only run the Path_redundant procedure on those constraints which, once removed, could result in the problem being reduced to an easier problem.

As mentioned before, one may attempt to remove certain constraints in order to reduce the problem to one which is decomposable into independent problems, or to one which constraint graph is a tree.

Domain knowledge may be useful in making such judgements.

7.5 Cycle-cutsets, Stable Sets and Pseudo_Tree_Search

7.5.1 The cycle-cutset method

The *cycle-cutset method* is a dynamic search method which can be applied to binary CSPs. (Although extending this method to general CSPs is possible, one may not benefit very much from doing so.)

The goal is to reduce the time complexity of solving the problem to below $O(a^n)$, where n is the number of variables and a is the maximum domain size in the problem.

The basic idea is to identify a subset of variables in the problem where removal will render the constraint graph being acyclic.

In general, the cycle-cutset method is useful for problems where most variables are constrained by only a few other variables.

Definition 7-1:
The **cycle-cutset** of a graph is a subset of the nodes in the graph which once removed, renders the graph as acyclic:

$\forall$ graph((V, E)): $\forall\ S \subseteq V$: cycle-cutset(S, (V, E)) $\equiv$ acyclic($(V - S, E')$)

where $E' = \{(a, b) \mid (a, b) \in \mathrm{E} \wedge a, b \in V - S)\}$ ■

There is no known efficient algorithm for finding the minimum cycle-cutset. One heuristic to find such sets is to use the reverse of a minimum width ordering or a maximum cardinality ordering. A less laborious way is to order the variables by their degrees in descending order instead of using the minimum width ordering.

The CCS procedure (CCS stands for Cycle Cut-Set) shows one possible way of using the cycle-cutset concept to solve binary CSPs.

```
PROCEDURE CCS(Z, D, C)
BEGIN       /* Ordering is an array 1..|Z| of nodes */
    /* preprocessing — identify a cycle-cutset */
    Ordering ← elements in Z ordered by descending order of their
        degrees in the constraint graph;
    Graph ← constraint graph of (Z, D, C);
    Cutset ← { }; i ← 1;
    WHILE (the graph of (Z, D, C) is cyclic) DO
        BEGIN
            Cutset ← Cutset + {Ordering[i]};
            remove Ordering[i] and edges involving it from Graph;
            i ← i + 1;
        END

    /* labelling */
    CL1 ← compound label for variables in Cutset satisfying all con-
        straints;
    REPEAT
        FOR j ← i to |Z| DO
            remove from D_Ordering[j] values which are incompatible with
                CL1;
        Achieve DAC in the remaining problem;
        IF (the remaining problem is 1-satisfiable) THEN
            BEGIN
                CL2 ← label the remaining variables using a backtrack-
                    free search;
                return(CL1 + CL2);
            END
        ELSE CL1 ← alternative consistent compound label for the
            Cutset, if any;
    UNTIL (there is no alternative consistent compound label for the
        Cutset);
    return(NIL);
END /* of CCS */
```

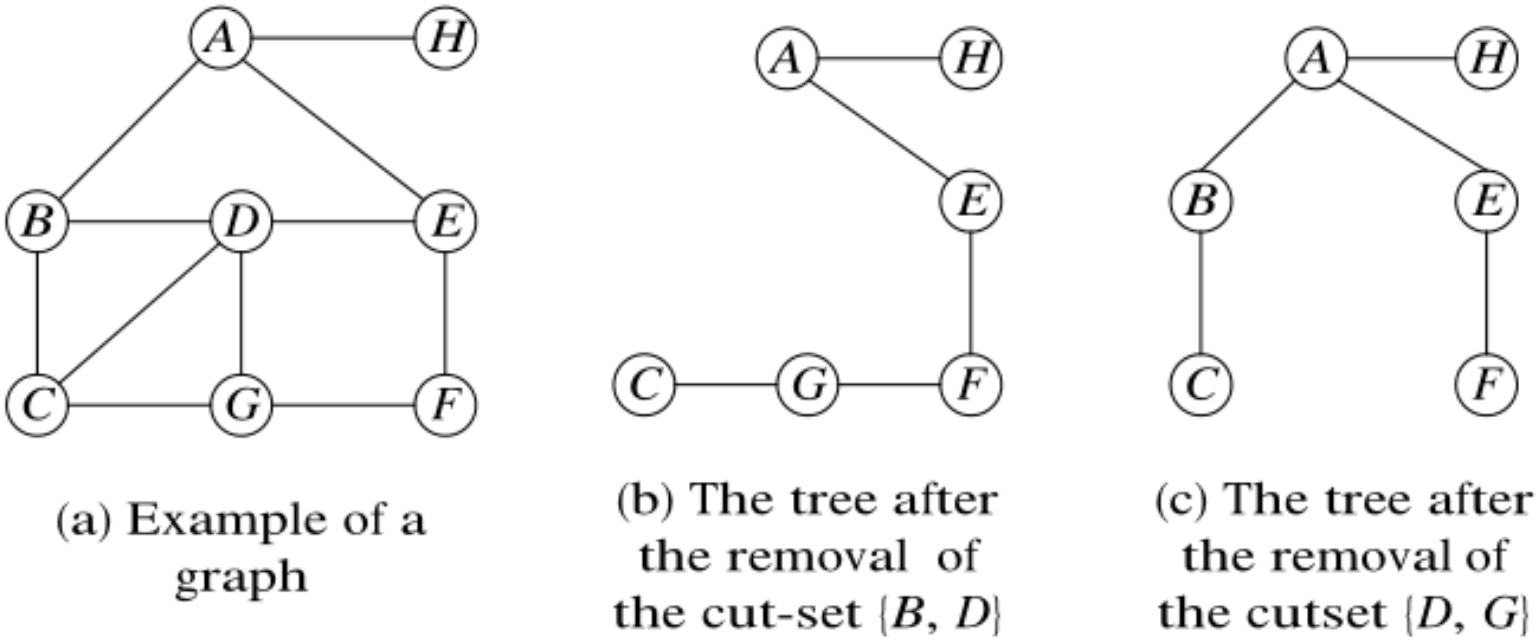


Figure 7.3 Examples of cycle-cutset

The cycle-cutset method does not specify how the problem in the cutset should be solved — this is reflected in the CCS procedure. Besides, orderings other than the one used in the CCS procedure can be used.

The acyclicity of a graph with n nodes can be determined in $O(n)$ using the Acyclic procedure. The preprocessing part of the algorithm takes $O(n^2)$ time to complete. In the worst case, the time complexity of finding a consistent compound label CL1 is $O(a^c)$ time, where a is the size of each domain, and c is the number of variables in the Cutset. CL2 is found using the Tree_search procedure. Therefore, the time complexity of finding CL2 is $O((n - c)a^1)$, or $O(na^2)$ in the worst case. The overall complexity of the procedure CCS is therefore $O(na^{c+2})$.

The cycle-cutset method can be used together with search strategies which use dynamic ordering. However, using the cycle-cutset method with such search strategies incurs the overhead of checking, after labelling every variable, whether the constraint graph of the unlabelled variables is acyclic.

[1] . The cycle-cutset method has been tested on *small* randomly generated CSPs (with maximum 15 variables and 9 values each), and is shown to out-perform BT (Section 5.2.1, Chapter 5) by 20% in terms of consistency checks [DecPea87]. Such a result does not give much support to the efficiency of this method in any realistic applications.

The cycle-cutset method can be extended to handle general constraints. This can be done by generating the primal graph before identifying a cycle-cutset. However, this would mean that whenever a k-ary constraint C is present, the cutset must contain at least $k - 2$ of the variables involved in C.

When the cycle-cutset method is used together with chronological backtracking, a smaller space will normally be searched, due to the use of DAC after the cycle cutset is labelled.

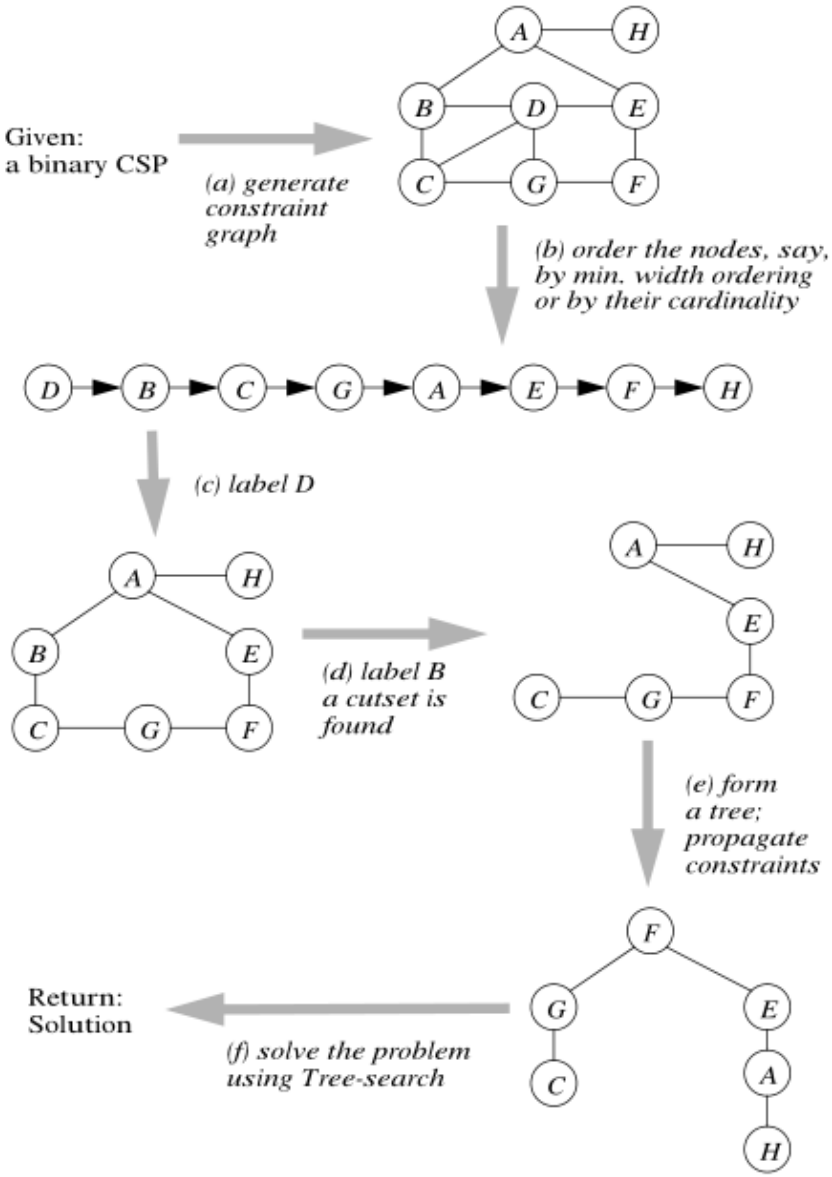


Figure 7.4 Procedure of applying the cycle-cutset method to an example CSP

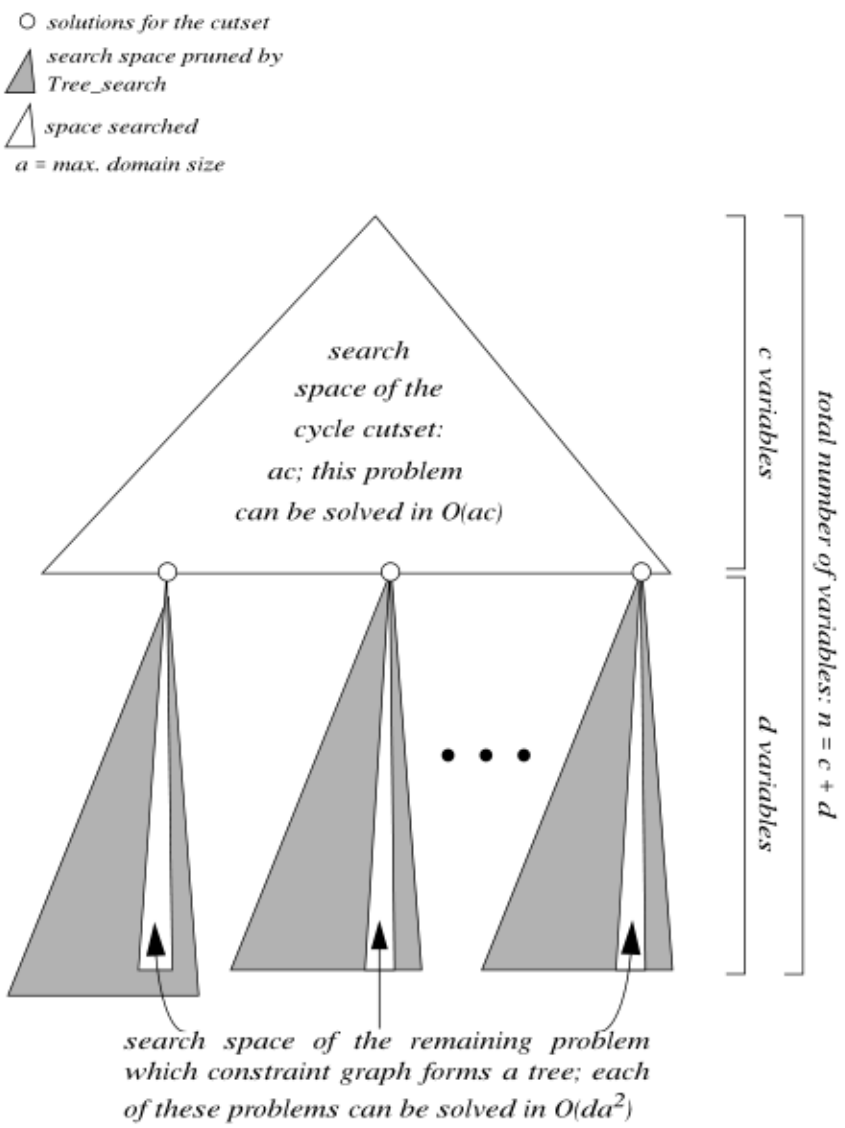


Figure 7.5 Search space of the cycle-cutset method. The overall complexity of the problem is $O(da^{c+2})$, where a is the maximum domain size, c is the size of the cycle cutset, and d is the number of variables not in the cycle cutset

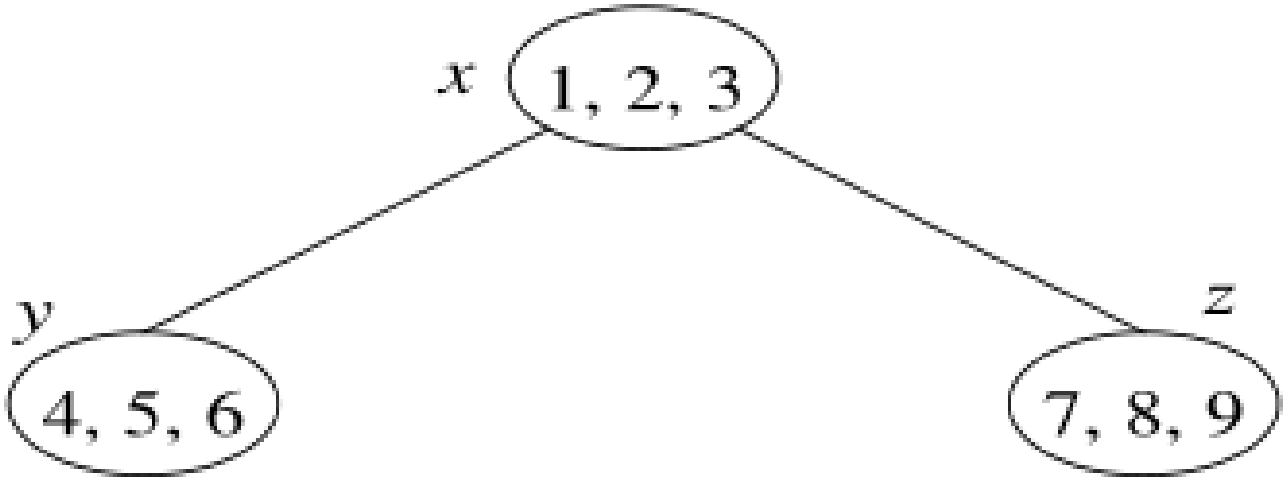
7.5.2 Stable sets

One may also exploit the topology of the primal graph by identifying stable sets. The principle is to partition the nodes in the primal graph into sets of mutually independent variables, so that they can be tackled separately.

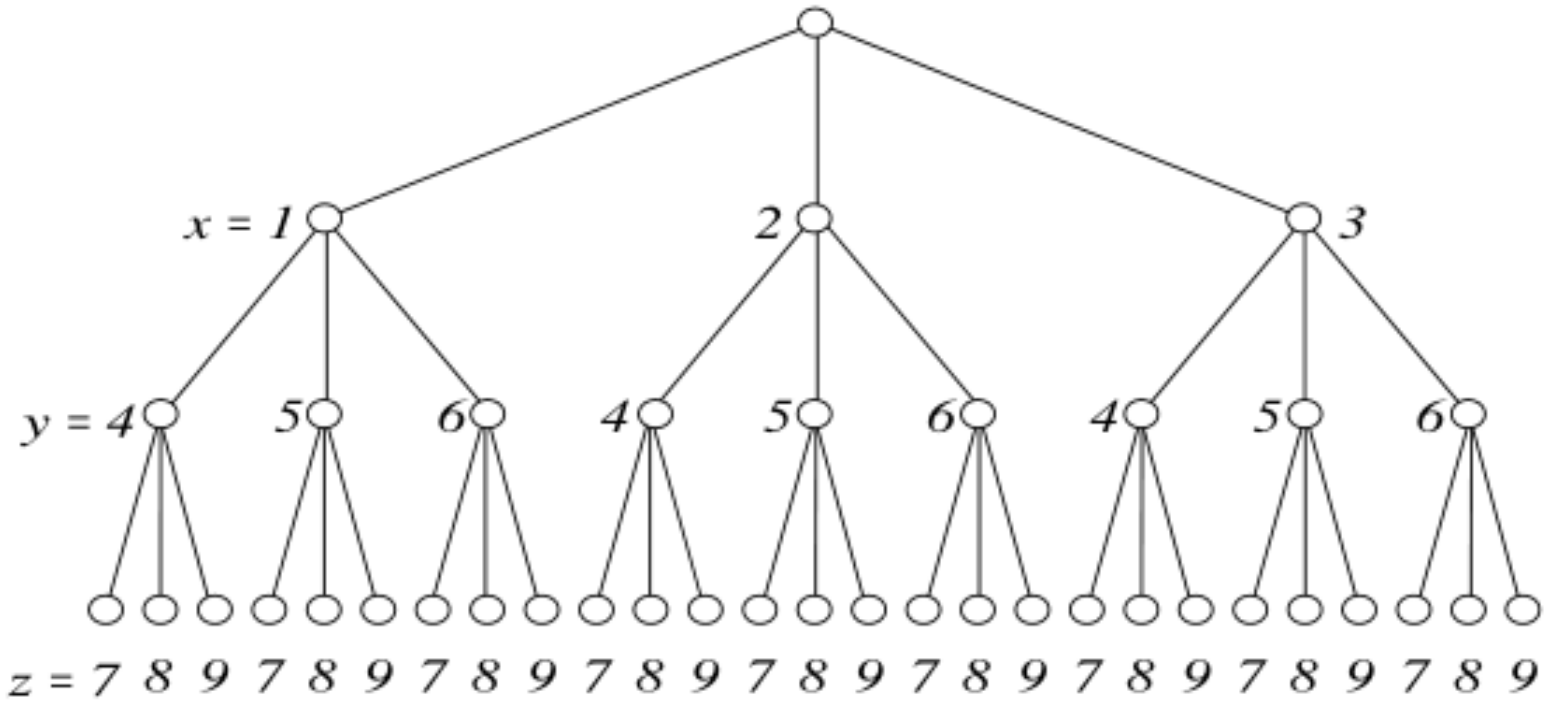
Definition 7-2:
A **stable set** S of a graph G is a set of non-overlapping sets of nodes in G such that no edge joins any two elements of different sets in S:

$$\forall\ \text{graph}((V, E)):\ \forall\ S \subseteq \{s \mid s \subseteq V\}:$$
$$\text{stable_set}(S, (V, E)) \equiv$$
$$(\forall\ s_1, s_2 \in S: (s_1 \cap s_2 = \{\} \wedge (\forall\ x \in s_1, y \in s_2: (x, y) \notin E))) \blacksquare$$

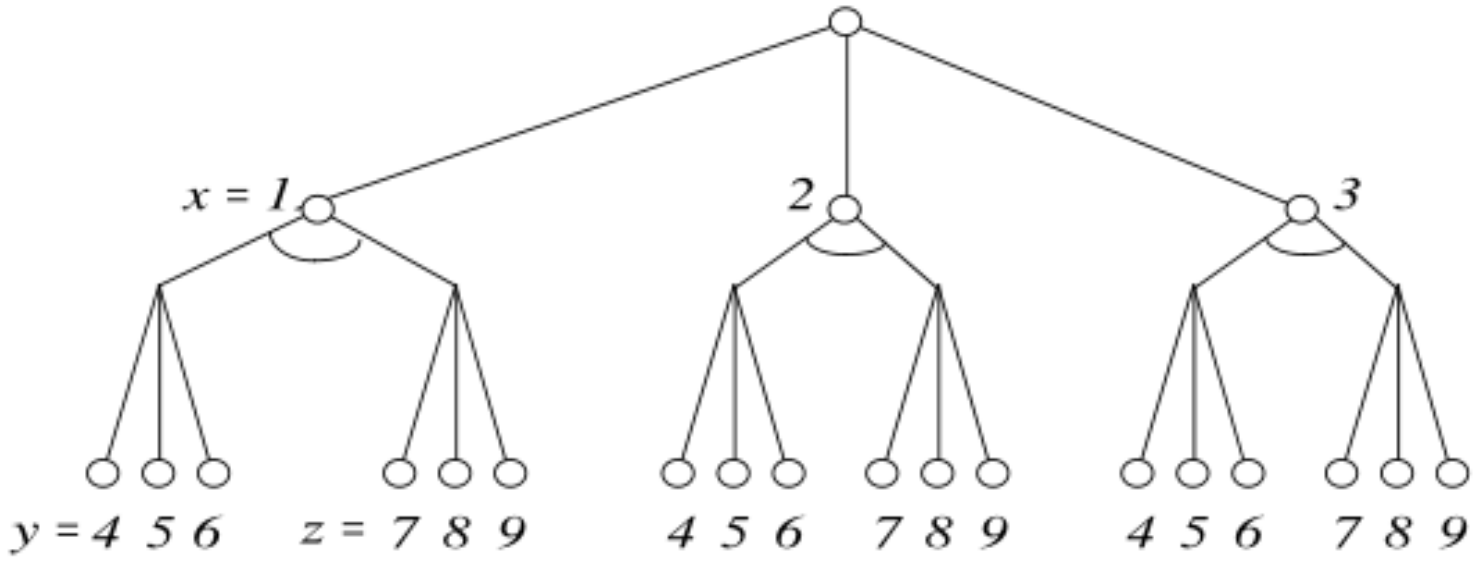
Figure 7.6 Example illustrating the possible gain in exploiting stable sets



(a) Example of a constraint graph



(b) Search space for the CSP in (a) under simple backtracking

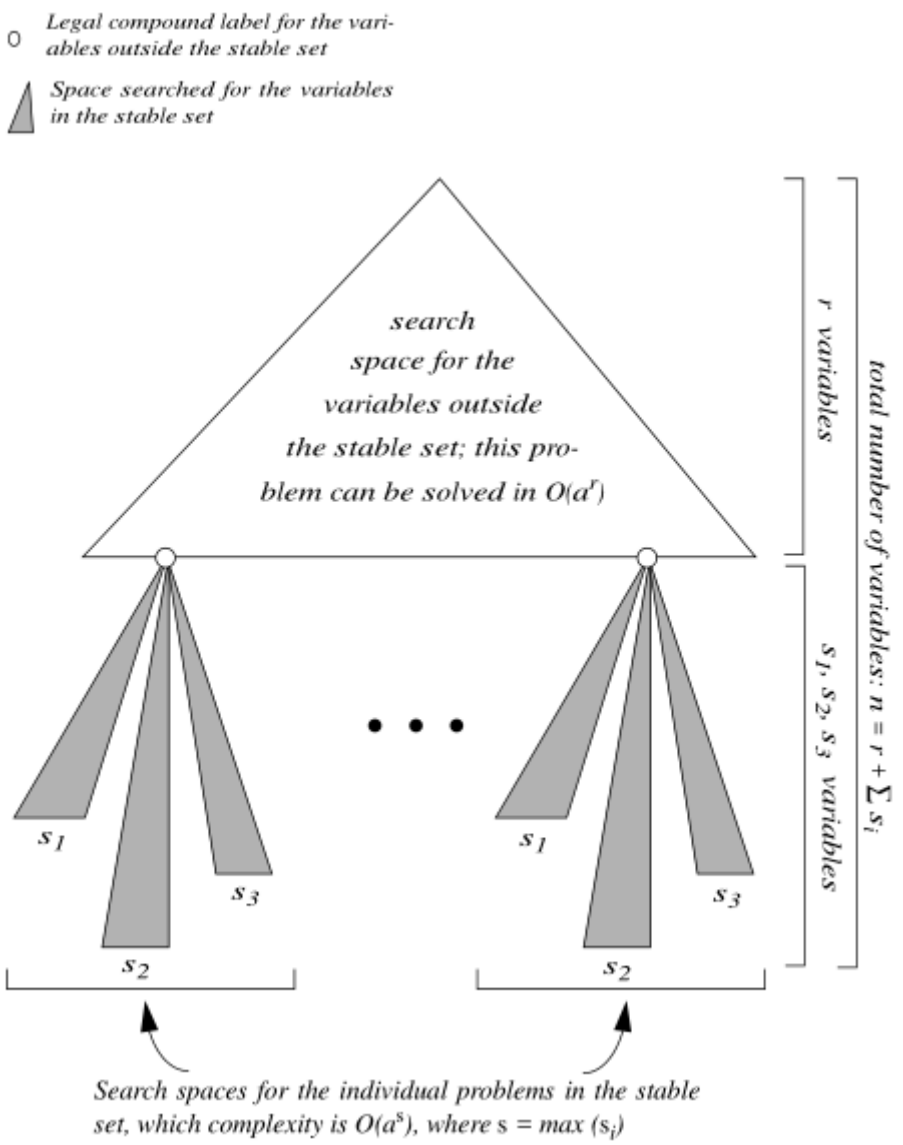


(c) Search space for the CSP in (a) when stable set is considered
(the search space forms an AND/OR tree)

In general, the sets in a stable set for a CSP may contain more than one variable. Assume that the variables are ordered in such a way that after labelling r variables, the rest of the variables can be partitioned into clusters that form a stable set. After a legal compound label CL for the r variables is found, the clusters can be treated as separate problems. If there exists no compound label for the variables in any of these clusters which is compatible with CL, then CL is discarded and an alternative compound label is found for the r variables. This process continues until compatible compound labels are found for the r variables and the individual clusters.

```
PROCEDURE Stable_Set(Z, D, C)
/* Given a CSP, Stable_Set returns a solution tuple if it is found; NIL
    otherwise; how the stable set SS is found is not suggested here */
CONSTANT r;
BEGIN
    /* initialization */
    Ordering ← an ordering of the variables in Z;
    R ← the set of the first r variables in the Ordering;
    SS ← stable set containing the rest of the variables partitioned;

    /* labelling starts */
    CL ← legal compound label for the variables in R;
    REPEAT
        FOR each S_i in SS DO
            Label_i ← compound label for S_i which is compatible with
                CL;
        IF (Label_i for some i is not found)
        THEN CL ← alternative legal compound label for the variables
            in R, if any ;
        ELSE return(SS + Label_i for all i);
    UNTIL (there is no alternative legal compound label for the varia-
        bles in R);
    return(NIL); /* signifying no solution */
END /* of Stable_Set */
```


Legal compound label for the variables outside the stable set
Space searched for the variables in the stable set
search space for the variables outside the stable set; this problem can be solved in $O(a^r)$
r variables
s_1, s_2, s_3 variables
total number of variables: $n = r + \sum s_i$
s_1
s_3
s_2
s_1
s_3
s_2
Search spaces for the individual problems in the stable set, which complexity is $O(a^s)$, where $s = max(s_i)$

For simplicity, we assume that the complexity of ordering the variables and partitioning the graph are relatively trivial compared with the labelling part of the algorithm.

The worst case time complexity for finding a legal compound label *CL* for the *r* variables is in the general $O(a^r)$, where *a* is the maximum domain size in the problem. Let the size of the *i*-th cluster be s_i, and *s* be the maximum value of all s_i. The complexity for finding a legal compound label for all clusters is in general $O(a^s)$. Therefore, the complexity of the whole problem is $O(a^{r+s})$.

Unfortunately, there is no known algorithm for ordering the variables so as to minimize $r + s$. The algorithm Stable_Set above does not specify how the variables should be ordered. One may choose different sizes (*r*) for the set *R* in the Stable_Set procedure above. The maximum size of the stable set (*s*) may vary depending on the Ordering and the *r* chosen.

7.5.3 Pseudo-tree search

Another algorithm which uses a similar idea as the stable sets is the pseudo-tree search algorithm. It uses a similar principle as the graph-based backjumping algorithm. When a variable x cannot be given any label which is compatible with the compound label committed to so far, both pseudo-tree search and graph-based backjumping will backtrack to the most recent variable y which constrains x.

The major difference is that when backtracking takes place, Graph-based Back-Jumping will undo all the labels given to the variables between y and (including) x. Pseudo-Tree Search will only undo those labels which are constrained by y. The overhead for doing so is the maintenance of the dependency relationship among the variables.

```
PROCEDURE Pseudo_Tree_Search(Z, D, C);
/* for simplicity, we assume that there are n variables and all their
    domains have size m; after ordering, let Z[i] be the i-th variable,
    D[i,j] be the j-th value of variable Z[i], v[i] be an index to a value in
    the domain of Z[i] */
BEGIN
    /* initialization */
    Order the variables in Z;
    Order the values in every domain in D;
    FOR i = 1 to n DO v[i] ← 1;   /* assign first value to each variable */
    i ← 1;
    /* searching */
    WHILE (i ≤ n) DO
        /* invariance: compound label for variables Z[1] to Z[i-1] is
            legal, and v[i] is an index to a value in the domain of Z[i]
            which is yet to be examined */
        BEGIN
            IF (legal(<Z[i], D[i,v[i]]>))
            THEN i ← i + 1;
            ELSE REPEAT
                    IF (v[i] < m)
                    THEN v[i] ← v[i] + 1;
                            /* give Z[i] an alternative value */
                    ELSE BEGIN /* backtrack */
                        p ← bt_level(i);       /* to be explained in text */
                        IF (p > 0)
                        THEN FOR k = p + 1 to i DO
                                IF (Z[k] is descendent of Z[p])
                                THEN v[k] ← 1;
                                /* v[p] is to be changed */
                        ELSE return(NIL);    /* no solution */
                        i ← p;     /* backtrack to variable Z[p] */
                    END
                UNTIL (legal(<Z[i],D[i,v[i]]>) OR (i < 1));
        END; /* of WHILE */
    IF (i < 1) THEN return(NIL)
    ELSE return(values indexed by v);
END /* of Pseudo_Tree_Search */
```

- For all i, $v[i]$ stores an index to the current value assigned to the variable $Z[i]$.
- The function legal(<$Z[i]$, $D[i,v[i]]$>) returns True if the label <$Z[i]$, $D[i,v[i]]$> is compatible with all the labels <$Z[h]$, $D[h,v[h]]$> for all $h < i$.
- If all the values of $Z[i]$ are incompatible with some labels committed so far, then the function *bt-level(i)* returns the greatest index j such that $Z[j]$ precedes $Z[i]$, and $Z[j]$ constrains $Z[i]$; 0 will be returned if no such $Z[j]$ exists. On the other hand, if some value for $Z[i]$ is compatible with all the labels committed to so far (i.e. $Z[i]$ has been successfully labelled but now it is backtracked to), then *bt_level*(i) returns $i-1$.
- $Z[k]$ is a descendent of $Z[p]$ if (i) $C_{Z[p], Z[k]} \in C$ and $p < k$; or (ii) $Z[k]$ is a descendent of any descendent of $Z[p]$. When $Z[p]$ is backtracked to, a new value will be given to it. Therefore, all the values for $Z[k]$ have to be considered; hence the first value is given to $Z[k]$.
- The Pseudo_Tree_Search procedure can be seen as a procedure which uses the stable set idea: when backtracking, variables are divided into two sets: those which require revision and those which do not.

7.6 The Tree-clustering Method

The tree-clustering method is useful for general CSPs in which every variable is constrained by only a few other variables. The tree-clustering method involves decomposing the problem into subproblems, solving the subproblems separately, and using the results to generate overall solutions. Interestingly, this process involves both adding and removing redundant constraints. The basic idea, which comes from database research, is to generate from the given CSP a new binary CSP whose constraint graph is a tree. Then this generated problem can be solved using the tree-searching technique introduced in Section 7.2. The solution of this generated CSP is then used to generate a solution for the original problem.

7.6.1 Generation of dual problems

7.6.2 Addition and removal of redundant constraints

7.6.3 Overview of the tree-clustering method

7.6.4 Generating chordal primal graphs

7.6.5 Finding maximum cliques

7.6.6 Forming join-trees

7.7 *j*-width and Backtrack-bounded Search

Theorem 6.1 states the relationship between k-consistency in a CSP and the width of its graph. We extend the concept of width to j-width, and show that it has interesting results related to (i, j)-consistency.

Definition 7-17:
A backtracking search for solutions in a CSP is called ***b*-level backtrack-bounded**, or ***b*-bounded** for simplicity, under an ordering if, after labelling h variables for any h less than the number of variables in the problem, we can always find a value for the (h + 1)-th variable without reconsidering more than the last $b - 1$ labels:

$\forall$ csp((Z, D, C)): ($\forall$ <: total_ordering(Z, <): ($\forall$ $b < |Z|$:
b-level-backtrack-bounded((Z, D, C), <) $\equiv$
($\forall$ $x_1, x_2, ..., x_h \in Z$: ($x_1 < x_2 < ... < x_h \Rightarrow$

$$
\begin{aligned}
&(\forall\ v_1 \in D_{x1},\ v_2 \in D_{x2},\ \ldots, v_h \in D_{xh}: \\
&\quad (\text{satisfies}(\ (\langle x_1, v_1\rangle \ldots \langle x_h, v_h\rangle), \text{CE}(\{x_1, \ldots, x_h\}, (Z, D, C))) \Rightarrow \\
&\quad\quad (\forall\ x_{h+1} \in Z: (x_h < x_{h+1} \Rightarrow \\
&\quad\quad\quad \exists\ v'_{h-b+1} \in D_{xh-b+1},\ \ldots, v'_h \in D_{xh},\ v_{h+1} \in D_{xh+1}: \\
&\quad\quad\quad\quad \text{satisfies}((\langle x_1, v_1\rangle \ldots \langle x_{h-b}, v_{h-b}\rangle\langle x_{h-b+1}, v'_{h-b+1}\rangle \ldots \\
&\quad\quad\quad\quad \langle x_h, v'_h\rangle \langle x_{h+1}, v_{h+1}\rangle), \text{CE}(\{x_1, \ldots, x_h, x_{h+1}\}, (Z, D, C))) \ \blacksquare
\end{aligned}
$$

Note that the *b* in the definition of *b-level-backtrack-bounded*, or *b-bounded,* is actually treated as an argument of the predicate (like the *k* in *k-consistency* in Chapter 3). A more accurate syntax which conforms to first order logic would be to put *b* between the brackets, which makes *b-level-backtrack-bound(b, Compound_label, C_s)*. The present syntax is adopted for both simplicity and conformation with the CSP literature. The same arrangement applies to the definition of *j*-width later in this chapter.

In other words, in a chronological backtracking search where b-bounded is guaranteed, if one can successfully label h variables without violating any constraints, then one can freeze the first $(h - b)$ labels in labelling the rest of the variables. A backtrack-free search is 0-bounded by definition.

Definition 7-18:
The **width of a group of j consecutive nodes** in a graph under an ordering is the number of nodes preceding this group which are joined to any of the j nodes in it:

$$\forall\ \text{graph}((V, E)): (\forall <: \text{total_ordering}(V, <):$$
$$(\forall\ x_1, x_2, ..., x_j \in V: \text{consecutive}((x_1, x_2, ..., x_j)):$$
$$\text{(width}((x_1, x_2, ..., x_j), (V, E), <) \equiv$$
$$|\{z \mid z \in V \wedge z \notin \{x_1, x_2, ..., x_j\} \wedge \exists\ w: (w \in \{x_1, x_2, ..., x_j\} \wedge z < w \wedge (z, w) \in E)\}|)))$$

where

$\forall$ graph((V, E)): ($\forall$ <: total_ordering(V, <): ($\forall$ $x_1, x_2, ..., x_j \in V$:

consecutive($(x_1, x_2, ..., x_j)$) $\equiv x_1 < x_2, ..., x_{j-1} < x_j \wedge$

($\forall$ $y \in V$: ($\exists$ $1 \leq i \leq j$: $y < x_i$) $\Rightarrow$ $y < x_1$) $\wedge$

($\forall$ $z \in V$: ($\exists$ $1 \leq i \leq j$: $x_i < z$) $\Rightarrow$ $x_j < z$))) ■

Definition 7-19:

The ***j*-width of a node** x is the minimum of the widths of all the groups of j or less consecutive nodes which end with x:

$\forall$ graph((V, E)): ($\forall$ <: total_ordering(V, <): ($\forall$ $x_m \in V$:

j-width(x_m, (V, E) , <) $\equiv$ MIN width($(x_{m-k+1}, ..., x_{m-1}, x_m)$, ($V$, E), <): $1 \leq k \leq j$)) ■

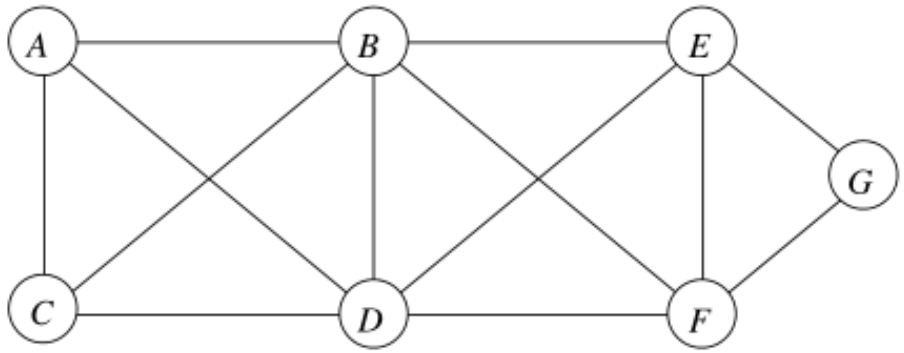
The concept j-width is a generalization of the concept width. According to this definition, the definition of *the width of a node* is equivalent to the *1-width of a node*.

Definition 7-20:
The ***j*-width of a graph under an ordering** is the maximum *j*-width of all the nodes in the graph under that ordering:

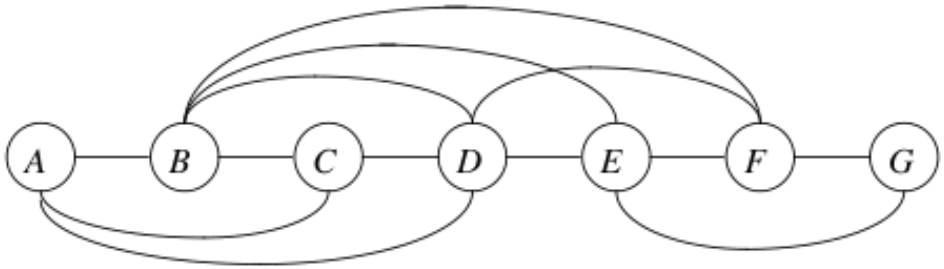
$\forall$ graph((*V*, *E*)): ($\forall$ <: total_ordering(*V*, <):
j-width(*V*, *E*), <) $\equiv$ MAX *j*-width(*x*, (*V*, *E*), <): $x \in V$ ■

Definition 7-21:
The ***j*-width of a graph** is the minimum *j*-width of the graph under all possible orderings of its nodes:

$\forall$ graph((*V*, *E*)): *j*-width((*V*, *E*)) $\equiv$ MIN *j*-width((*V*, *E*), <): total_ordering(*V*, <) ■



(a) Example of a graph



1-width:	*0*	*1*	*2*	*3*	*2*	*3*	*2*	**3**
2-width:	*0*	*0*	*1*	*2*	*2*	*2*	*2*	**2**
3-width:	*0*	*0*	*0*	*1*	*2*	*2*	*2*	**2**

(b) j-width of the nodes for $1 \leq j \leq 3$, given the ordering A, B, C, D, E, F, G (the width of each node is shown in italic, the j-width of the orderings are indicated in bold)

Figure 7.14 Example of a graph and the j-widths of an ordering

Theorem 7.6 (due to Freuder, 1985)
Given a constraint graph for a CSP, there exists a search order that guarantees j-bounded backtrack search if the graph is strong (i, j)-consistent for i equals to the j-width of the graph.

$$\forall\ \text{csp}((Z, D, C)): (\forall\ i, j \leq |Z|:$$
$$\quad (\text{strong } (i, j)\text{-consistent}(Z, D, C) \Rightarrow$$
$$\quad\quad (\forall <: \text{total_ordering}(Z, <):$$
$$\quad\quad\quad (i = j\text{-width}(\ G((Z, D, C)), <) \Rightarrow$$
$$\quad\quad\quad\quad j\text{-level-backtrack-bounded}((Z, D, C), <)))))$$

In other words, given a problem whose j-width is equal to i, one can determine the bound for one's backtrack search if one can maintain strong (i, j)-consistency for this problem. This implies that by finding an ordering which has the minimum j-width, one can minimize i in maintaining strong (i, j)-consistency.

The j-width of an ordered CSP can be determined by a branch and bound method. Unfortunately, maintaining strong (i, j)-consistency may change the width of the constraint graph.

Besides, there are no efficient algorithms for determining the j-width of an ordered CSP and maintaining (i, j)-consistency.

Although Theorem 7.6 is an interesting observation, its practical use in CSPs solving is yet to be explored.

7.8 CSPs with Binary Numerical Constraints

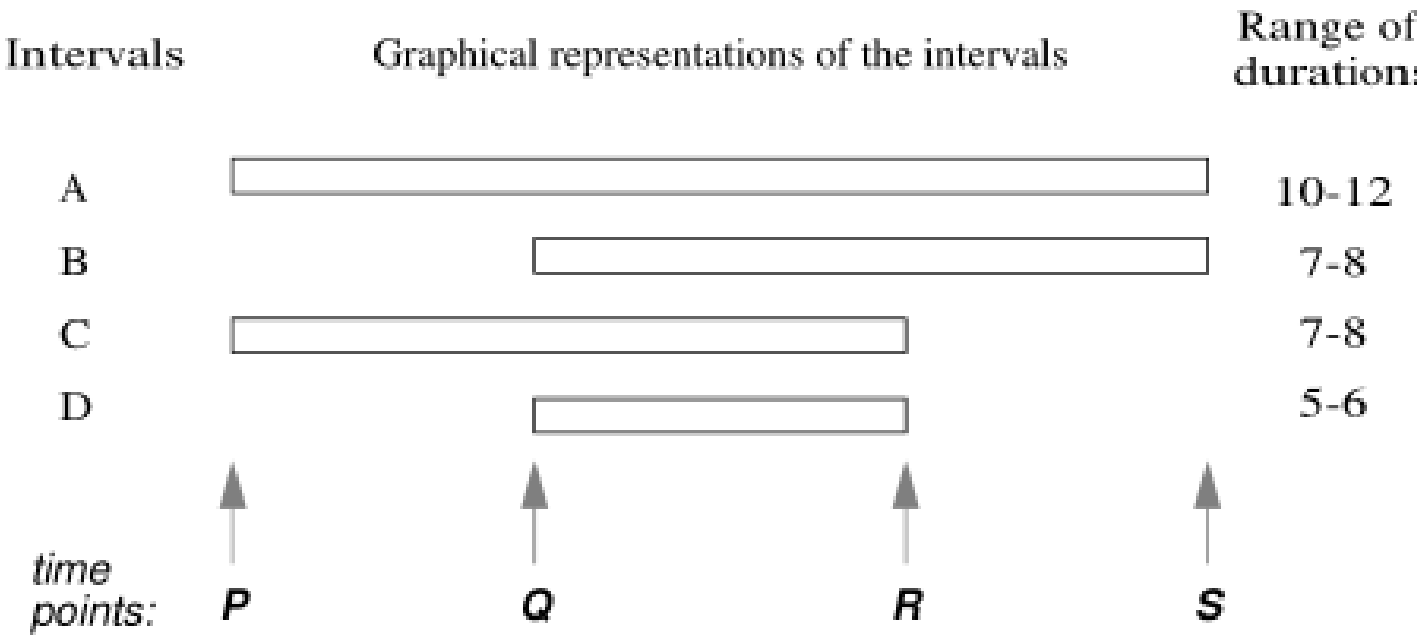
When all the variables in a CSP are numerical variables, and there exist unary and binary linear constraints only, specialized linear programming techniques can be applied. When variables are allowed to take numbers as their values, the problem is a non-standard CSP, because the domains are infinite. However, since the constraints take special forms, efficient algorithms exist for finding solutions for them.

Research in such problems is partly motivated by point-based temporal reasoning. In point-based temporal reasoning, time points are taken as primitive objects. Intervals may be represented by pairs of points. One of the tasks in temporal reasoning is to assign a numerical value to each time point, satisfying constraints on them.

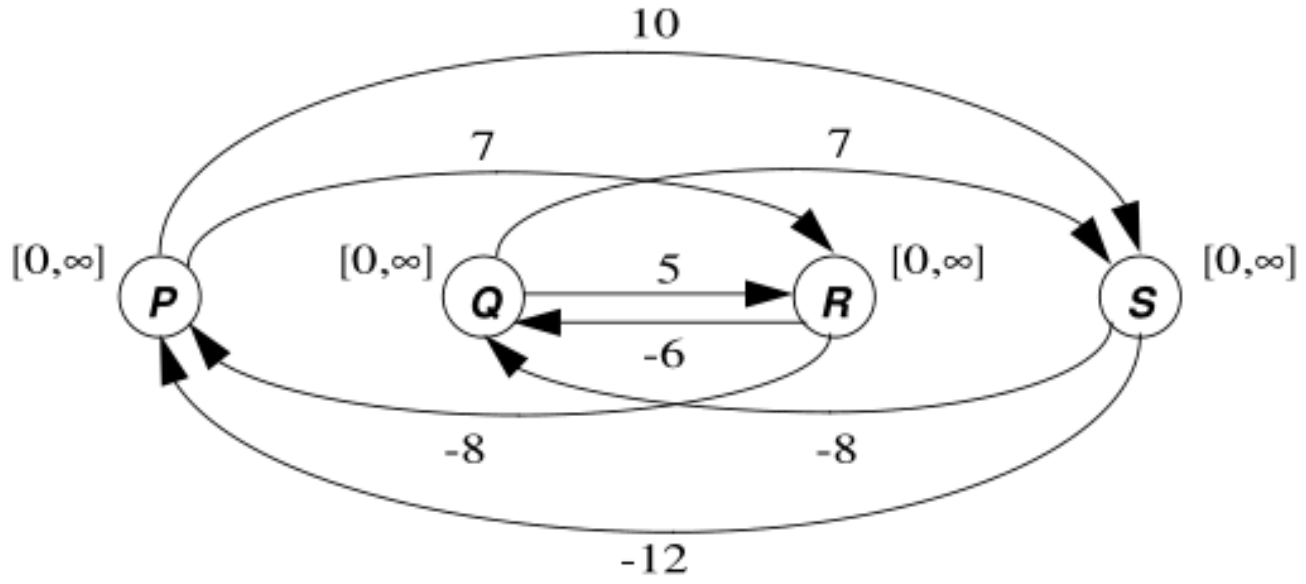
Simple temporal constraints are:

- **boundary constraints:** The value of a time point may be given a lower bound, which is called the *earliest time*, and a upper bound, which is called the *latest time*. In other words, given a time point x, there may be constants a and b such that $x > a$ *and* $x < b$ must hold, where < and > can also be ≤ and ≥.

- **distance constraints:** A *distance constraint* requires that the distance between two points be bounded within a range. For example, if x and y are variables representing two time points, and a and b are constants, a distance constraint may take the following form: $x - y > a$; $x - y < b$; $a < x - y < b$, where < and > can also be ≤ and ≥. Examples of distance constraints are upper bounds and lower bounds on *durations*. A *precedence constraint* is a special kind of distance constraint where the constants are 0. In other words, precedence constraints take the form: $x < y$



(a) Example set of intervals, points and constraints (assuming no unary constraints on the time points)



(b) Temporal constraint graph of the time points in (a) (the lower- and upper-bounds of the nodes are shown in [ ]'s)

Figure 7.15 Example of a set of constrained intervals and points and their corresponding temporal constraint graph

```
PROCEDURE AnalyseLongestPaths(V, E, length, lower_bound)
/* (V, E) is a directed graph; length(c) returns the length of an arc c ;
    lower_bound(p) returns the earliest starting time of point p in V;
    x[i] stores the updated lower bound for point i in V; */
/* AnalyseLongestPaths labels all x[i] */
BEGIN
    FOR each i in V DO x[i] ← lower_bound(i);
    Converged ← False;
    Counter ← 0;
    WHILE (NOT Converged) DO
        BEGIN
            Converged ← True;
            FOR j = 1 to |Z| DO
                FOR each k such that j→k is in E DO
                    IF (x[k] < x[j] + length(j→k)) THEN
                        BEGIN
                            x[k] ← x[j] + length(j→k);
                            Converged ← False;
                        END
            Counter ← Counter + 1;
            IF Counter > |Z| THEN return(NIL);   /* over-constrained */
        END
    /* on exit of the WHILE loop, all the constraints are satisfied */
    return(x);          /* x is the array of all the variables */
END /* of AnalyseLongestPaths */
```

Given a temporal constraint graph, the AnalyseLongestPaths algorithm checks if temporal constraints are satisfiable, and if so, returns the earliest possible times for each of the time points in the graph.

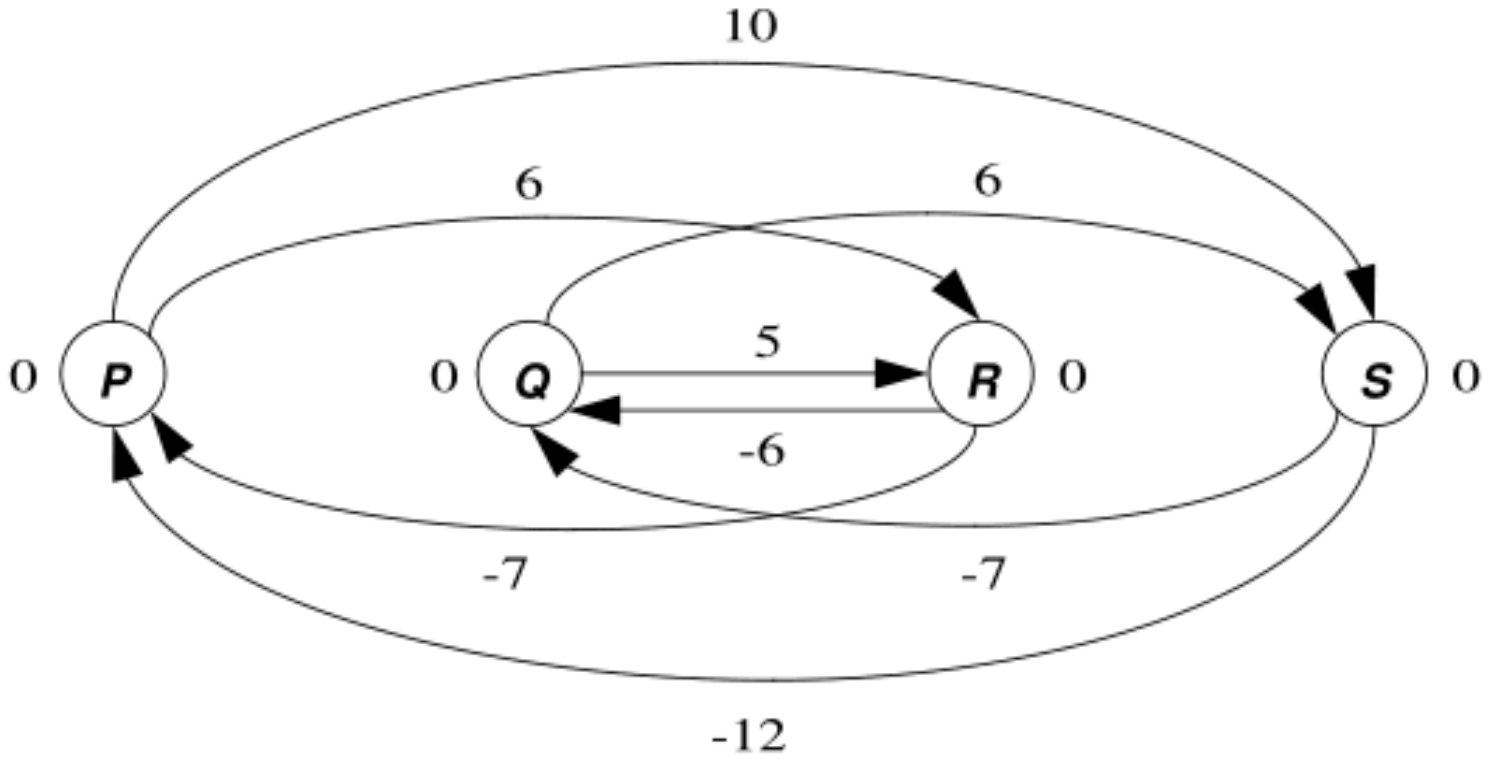
A constraint should never be propagated more than n times, where n is the number of nodes in the graph. If this happens, it indicates that the value of a node is updated because of its own update. In this case, one can conclude that the constraints are not satisfiable. The Counter helps us to detect such situations. The WHILE loop terminates when no lower bound has been updated.

The AnalyseLongestPaths algorithm finds the longest possible distance from every node to its successor nodes in the graph (hence its name). The AnalyseLongestPaths algorithm finds (or updates) the lower bounds of each node in the graph. It can be modified to the AnalyseShortestPaths algorithm which finds the upper bounds of the nodes.

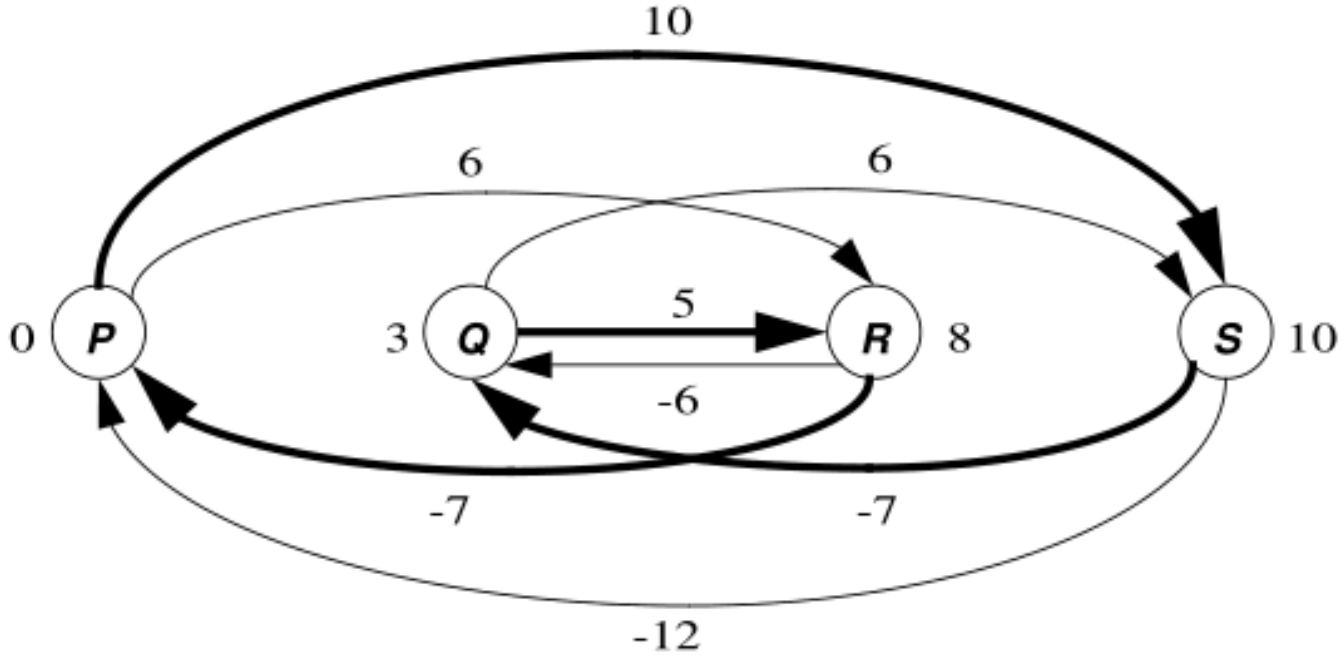
```
PROCEDURE AnalyseShortestPaths(V, E, length, upper_bound)
/* (V, E) is a directed graph; length(c) returns the length of an arc c;
    upper_bound(p) returns the latest starting time of point p in V;
    y[i] stores the updated upper bound for point i in V; */
/* AnalyseShortestPaths labels all y[i] */
BEGIN
    FOR each i in V DO y[i] ← upper_bound(i);
    Converged ← False;
    Counter ← 0;
    WHILE (NOT Converged) DO
        BEGIN
            Converged ← True;
            FOR j = 1 to |Z| DO
                FOR each k such that k→j is in E DO
                    IF (y[k] > y[j] – length(j→k)) THEN
                        BEGIN
                            y[k] ← y[j] – length(j→k);
                            Converged ← False;
                        END
             Counter ← Counter + 1;
             IF Counter > |Z| THEN return(NIL);   /* over-constrained */
         END
     /* on exit of the WHILE loop, all constraint have been satisfied*/
    return(y);          /* y is the array of all the variables */
END /* of AnalyseShortestPaths*/
```

After running both AnalyseLongestPaths and AnalyseShortestPaths on a temporal constraint graph, one may obtain both the lower bounds and the upper bounds for all the time points in the graph.

One nice property of the above two algorithms is that constraints can be added incrementally. After the upper and lower bounds of the points are computed, new constraints may be added to the constraint graph. Instead of computing the bounds from scratch, these algorithms may start with the values computed in the past so as to save computation.



(a) Example of an input temporal constraint graph to the Analyse-LongestPaths procedure — all lower bounds are initialized to 0



(b) The constraint graph after propagation of the constraints on the highlighted arcs once — a loop is found, hence the constraints are unsatisfiable

Figure 7.16 Example of an unsatisfiable temporal constraint graph detected by the AnalyseLongestPaths procedure

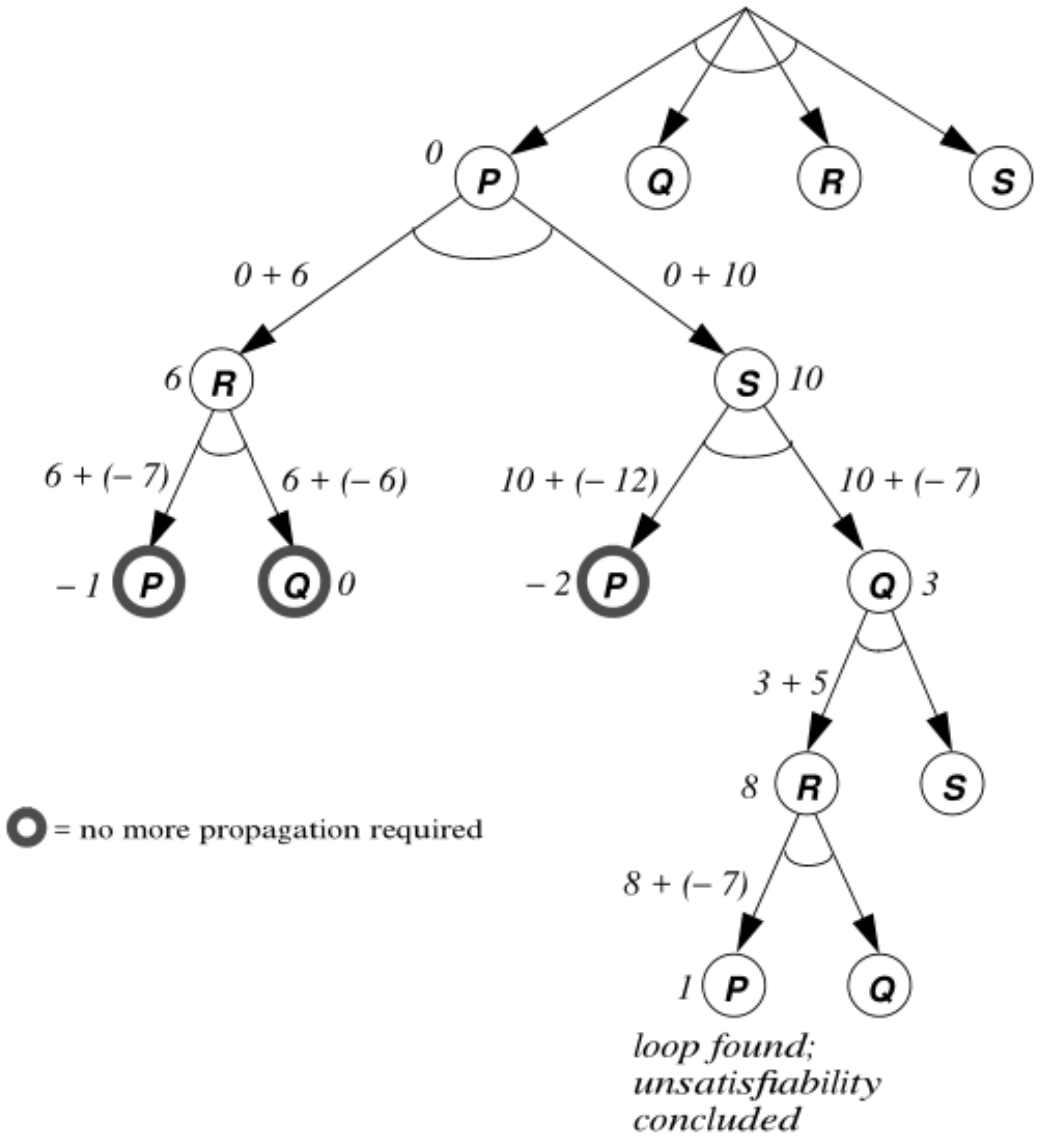


Figure 7.17 Possible space searched by AnalyseLongestPaths for the temporal constraint graph in Figure 7.16